

Netværks-lag

Byg og forstå din egen netværks-stack fra bunden

DNS, TCP, HTTP og HTTPS i en IT-sikkerhedskontekst

Arbejdsform: Individuel eller par · Miljø: Din Linux-VM fra Linux Basics-forløbet

Indholdsfortegnelse

Indholdsfortegnelse	2
1. Formål med opgaven.....	3
2. TCP, DNS, HTTP og HTTPS i en IT-sikkerhedskontekst	3
3. Læringsmål	4
4. Opgavens moduler	5
Modul 1: TCP — transportlaget	5
Hvorfor det er relevant i en sikkerhedskontekst	5
Miljø: din værtsmaskine vs. dit gæstesystem	5
Hvorfor hele opgaven kører på localhost i én VM	5
Sådan navngiver og adresserer du dine forbindelser	6
Sådan er din TCP-server og -klient bygget i Python	6
Firewall og netværksrettigheder i din VM	6
Wireshark i din VM.....	6
Opgaver	7
Afleveringskrav for modulet.....	7
Modul 2: DNS — fra navn til adresse	8
Hvorfor det er relevant i en sikkerhedskontekst	8
Opgaver	8
Afleveringskrav for modulet.....	8
Sådan bygger du DNS-opslagsscriptet i Python	8
Modul 3: HTTP — byg din egen webserver	9
Hvorfor det er relevant i en sikkerhedskontekst	9
Opgaver	9
Afleveringskrav for modulet.....	9
Sådan er din HTTP-server bygget i Python	9
Modul 4: HTTPS/TLS — kryptering af forbindelsen	10
Hvorfor det er relevant i en sikkerhedskontekst	10
Opgaver	10
Afleveringskrav for modulet.....	10
Sådan genererer du et selvsigneret certifikat.....	10
Sådan tilføjer du TLS til din server i Python	10
Modul 5: Integration — hele kæden samlet	11
Hvorfor det er relevant i en sikkerhedskontekst	11
Opgaver	11
Dokumentationskrav for modulet.....	11
Om browser-test (valgfrit).....	11
6. Samlet afleveringskrav	11

1. Formål med opgaven

Denne opgave samler de centrale netværksteknologier — TCP, DNS, HTTP og HTTPS — i ét sammenhængende, praksisnært projekt. I stedet for kun at observere protokollerne udefra (fx ved at kalde et færdigt bibliotek eller bruge en cloud-DNS-udbyder), skal du selv bygge de centrale dele af stakken fra bunden: din egen TCP-forbindelse, din egen navneopløsning, din egen HTTP-server og din egen TLS-krypterede forbindelse.

Formålet er, at du kommer til at forstå protokollerne indefra frem for kun at kunne bruge dem. Det giver dig et markant stærkere fundament for både at fejlfinde netværksproblemer og forstå, hvor og hvorfor sårbarheder typisk opstår i den stak, næsten al moderne software er bygget oven på.

Opgaven bygger direkte videre på Linux Basics-forløbet: du skal bruge den samme Linux-VM, du satte op og hærdede i SecureBase-opgaven, som dit arbejdsmiljø gennem hele opgaven.

Overordnet scenarie

- Du er ny som netværks- eller backend-udvikler i en lille tech-startup, der vil forstå deres egen infrastruktur i bund frem for blot at stole på færdige cloud-tjenester.
- Din opgave er at bygge en fungerende mini-webtjeneste fra bunden — egen navneopløsning, egen webserver, sikret med HTTPS — og kunne forklare og dokumentere hele kæden fra domænenavn til indhold i browseren.
- Du skal desuden kunne fejlfinde, når noget i kæden går i stykker, ved brug af de samme værktøjer som en netværkstekniker eller sikkerhedsanalytiker bruger dagligt.

2. TCP, DNS, HTTP og HTTPS i en IT-sikkerhedskontekst

TCP, DNS og HTTP(S) er ikke bare teknisk pensum — de er de lag, næsten al netværkstrafik passerer igennem, og derfor også nogle af de mest udnyttede angrebsflader i moderne IT-sikkerhed. At forstå dem indefra giver dig et fundament, du vil bruge i stort set enhver teknisk sikkerhedsrolle:

- TCP og statefulness: Firewalls, IDS/IPS og loadbalancere træffer beslutninger baseret på TCP's forbindelsestilstande. Angreb som SYN-flood udnytter netop denne mekanik, og at kunne læse en handshake i programmer som Wireshark er en grundlæggende færdighed i netværksfejlfinding og -overvågning.
- DNS som angrebsflade: DNS er ofte usikret og et hyppigt mål for spoofing og cache poisoning. Det bruges også aktivt af angribere til data-eksfiltrering og command-and-control-kommunikation (DNS tunneling), fordi DNS-trafik sjældent bliver nøje inspiceret.
- HTTP i klartekst: HTTP sender alt — inklusive potentielt følsomme data — i læsbar tekst. At se det med egne øjne i en pakke-capture gør det konkret, hvorfor aldrig at sende følsomme oplysninger over almindelig HTTP.
- HTTPS/TLS som tillidsfundament: TLS er grundlaget for tillid på hele internettet i dag. Forkert konfigureret TLS (udløbne certifikater, forældede protokolversioner) er blandt de hyppigste fund i sikkerhedsscanninger, og certificate trust chain er det, der i praksis forhindrer man-in-the-middle-angreb.
- Værktøjer du vil bruge resten af din karriere: `dig/nslookup`, `curl -v`, `openssl s_client`, `netstat/ss` og Wireshark er standardværktøjer for netværksteknikere, SOC-analytikere og pentestere — men de giver kun mening, hvis du forstår, hvad de faktisk viser dig.
- Byg-selv giver dybere forståelse: Ved selv at skrive en simpel HTTP-parser eller sende en DNS-query manuelt over UDP oplever du direkte, hvor let det er at overse fejl i input-håndtering — den samme type fejl, der i virkelige systemer bliver til sårbarheder.

Opgaven giver dig derfor ikke kun 'kan bruge curl'-færdigheder, men en forståelse af, hvad der reelt sker under overfladen, hver gang en bruger taster en adresse i en browser.

3. Læringsmål

Efter at have gennemført opgaven kan du:

- Forklare og demonstrere TCP's three-way handshake og forbindelsestilstande.
- Bygge en simpel TCP-baseret klient og server ved brug af sockets.
- Forklare DNS-opslagsprocessen og de vigtigste recordtyper (A, CNAME, MX, NS, TXT).
- Konstruere og sende et DNS-opslag manuelt, samt konfigurere lokal navneopløsning til dit eget domænenavn.
- Bygge en simpel HTTP-server fra bunden, der parser rå requests og returnerer korrekte responses.
- Tilføje TLS/HTTPS til din webserver og forklare handshake og certificate trust.
- Anvende netværksanalyseværktøjer (Wireshark, curl, dig/nslookup, openssl s_client) til at dokumentere og fejlfinde trafik.
- Identificere, undersøge og udbedre en bevidst indbygget fejl et sted i din netværksskik.

De færdigheder, du træner i denne opgave, går igen direkte i flere IT- og sikkerhedsroller:

- Netværkstekniker/administrator: Fejlfinder dagligt forbindelsesproblemer på tværs af DNS, TCP og applikationslag, ofte under tidspres.
- Backend-/platformsudvikler: Forstår, hvad der reelt sker 'under' det webframework eller HTTP-bibliotek, der bruges i det daglige arbejde.
- SOC-analytiker: Bruger Wireshark og lignende værktøjer til at analysere mistænkelig netværkstrafik, herunder DNS-baserede angreb og unormale TLS-mønstre.
- Penetrationstester/etisk hacker: Udnytter ofte netop svagheder i DNS-opsætning, forældet TLS eller dårligt implementerede HTTP-servere — den viden du her bygger, er den samme, der bruges til at finde og udnytte disse svagheder etisk.

Målet med Opgaven er derfor ikke kun at gennemføre alle delopgaver, men at give dig en forståelse af netværksskikken, du kan tage direkte med ind i praktik eller job — uanset om du ender som udvikler, netværkstekniker eller sikkerhedsspecialist.

4. Opgavens moduler

Opgaven er delt i seks moduler, der bygges oven på hinanden — hvert lag i din stak bygger direkte videre på det forrige. Hvert modul indeholder et fagligt formål, en forklaring af den sikkerhedsmæssige relevans, konkrete opgaver samt afleveringskrav.

Modul 1: TCP — transportlaget

Formål: At forstå, hvordan pålidelig, forbindelsesorienteret kommunikation fungerer under HTTP, ved selv at bygge en simpel TCP-klient og -server.

Hvorfor det er relevant i en sikkerhedskontekst

- Firewalls og IDS/IPS-systemer træffer beslutninger baseret på TCP's forbindelsestilstande — at forstå handshake og state er en forudsætning for at kunne læse deres logs.
- Angreb som SYN-flood udnytter direkte TCP's forbindelsesopbygning, og kan kun genkendes, hvis du ved, hvordan en normal handshake ser ud.
- Værktøjer som netstat/ss, som bruges dagligt i drift og incident response, giver kun mening, hvis du forstår de underliggende forbindelsestilstande.
- At binde en tjeneste til et bestemt interface (fx kun loopback) frem for 'alle interfaces' er en grundlæggende sikkerhedsvane — mindst mulig eksponering er billigere at forsvare end at rydde op efter en tjeneste, der var tilgængelig for flere end tiltænkt.

Miljø: din værtsmaskine vs. dit gæstesystem

Denne opgave bruger to maskiner i ordets tekniske forstand, men kun én af dem er relevant for dig at arbejde i:

- Værtsmaskinen er din fysiske computer. Den kører VirtualBox og selve VM-vinduet, men du skal ikke installere, kode eller konfigurere noget her.
- Gæstesystemet er den Linux-VM, du satte op og hærkede i SecureBase-opgaven. Det er herinde, alt arbejde i netværks opgaven foregår: Python-kode, terminalkommandoer, Wireshark, hosts-filen og al test.

Tjek at du reelt arbejder i gæstesystemet, før du går videre: åbn en terminal inde i VM'en, og kørs `hostname` samt `ip a` (eller `hostname -I`). Ser du din VM's eget `hostname` og en IP-adresse, du selv konfigurerede i Linux Basics-forløbet, er du det rigtige sted. Medmindre andet er nævnt eksplicit, forudsætter resten af denne opgavebeskrivelse, at enhver kommando køres inde i gæstesystemet.

Hvorfor hele opgaven kører på localhost i én VM

I modsætning til DomainZero-opgaven skal du ikke bygge to VM'er eller konfigurere et virtuelt netværk mellem dem. Både din server og din klient kører som to separate processer i den samme VM, og de taler sammen via loopback-interfacet — adressen 127.0.0.1, også kendt som localhost. Det er en bevidst forenkling:

- Du får den fulde, ægte protokoladfærd — et rigtigt TCP-handshake, en rigtig DNS-forespørgsel, et rigtigt TLS-handshake — uden at skulle bekymre dig om DHCP, netværkskort eller en anden VM's tilstand.
- Der er ingen VirtualBox-netværksindstillinger at konfigurere; loopback findes altid, uanset hvordan din VM's netværkskort er sat op.
- Loopback-trafik forlader aldrig maskinens egen netværksstak og passerer derfor normalt slet ikke gennem eksterne firewall-regler eller institutionens netværk — det fjerner en hel kategori af fejlkilder, som ikke har noget med selve protokollerne at gøre.

Loopback-adresser ligger i hele IP-intervallet 127.0.0.0/8, men konventionen er at bruge præcis 127.0.0.1. Navnet localhost er allerede sat op til at pege på denne adresse i din VM's /etc/hosts-fil — det er faktisk præcis den samme mekanik, du selv skal bruge i modul 2 til dit eget domænenavn.

Sådan navngiver og adresserer du dine forbindelser

- Brug adressen 127.0.0.1 (eller navnet localhost) som mål, når din klient skal oprette forbindelse til din egen server.
- Bind altid din server til 127.0.0.1 og ikke til 0.0.0.0. 0.0.0.0 betyder 'lyt på alle netværksinterfaces', hvilket i praksis også ville gøre din server tilgængelig for andre på samme netværk som din VM — helt unødvendigt til denne opgave, og et godt eksempel på at vælge det mindst eksponerede setup fra start.
- Vælg et sigende variabelnavn eller en konstant i din kode for host og port (fx `HOST = "127.0.0.1"` og `PORT = 8080`), så det er tydeligt for en, der læser din kode, hvor serveren lytter.

Sådan er din TCP-server og -klient bygget i Python

Du skal bruge Pythons indbyggede socket-modul — ikke et højniveau-bibliotek som `http.server` eller `requests`, da hele pointen er at bygge det, de biblioteker ellers ville klare for dig. Nedenfor er den overordnede struktur, du skal implementere; du skal selv skrive koden.

Serverens opbygning: Opret et TCP-socket-objekt med `socket.socket(AF_INET, SOCK_STREAM)`. Bind det til din adresse og port med `.bind((HOST, PORT))`. Sæt det til at lytte efter forbindelser med `.listen()`. Byg derefter en løkke, der kalder `.accept()` — dette kald blokerer, indtil en klient forbinder, og returnerer et nyt socket-objekt for netop den forbindelse. Brug dette nye objekt til at læse data med `.recv(bufferstørrelse)` og svare med `.sendall(data)`. Luk forbindelsen med `.close()`, gerne ved hjælp af en `with`-kontekst, så den lukkes korrekt selv hvis noget fejler undervejs.

Klientens opbygning: Opret et socket-objekt på samme måde, kald `.connect((HOST, PORT))` for at oprette forbindelse til din server, send data med `.sendall()`, læs svaret med `.recv()`, og luk forbindelsen igen.

Bytes vs. tekst: Sockets arbejder udelukkende med bytes, ikke tekststreng. Husk derfor at `.encode()` dine strenge (fx til UTF-8) før du sender, og `.decode()` de bytes, du modtager, før du bruger dem som tekst.

Bufferstørrelse: Vælg en fornuftig bufferstørrelse til `.recv()`, fx 1024 byte. Til de simple beskeder i dette modul er det rigeligt, men det er værd at bide mærke i, at `.recv()` ikke garanterer at modtage hele beskeden på én gang — noget du kan undersøge nærmere, hvis du vil gå i dybden.

Firewall og netværksrettigheder i din VM

Fordi al trafik i denne opgave forbliver på loopback-interfacet, er der som udgangspunkt intet at konfigurere: standardopsætningen i `ufw`/`iptables` tillader normalt trafik på `lo` uden yderligere handling fra din side. Har du lavet meget stramme, eksplicitte regler i SecureBase-opgaven, er det dog værd lige at tjekke med `sudo ufw status` eller `sudo iptables -L`, at loopback-interfacet ikke er blevet blokeret ved en fejl.

Wireshark i din VM

Da alt arbejde foregår i din Linux-VM, er Wireshark-opsætningen enkel sammenlignet med en Windows-installation (ingen Npcap-driver nødvendig):

1. Installér Wireshark i din VM: `sudo apt install wireshark`. Svar 'Yes', hvis du bliver spurgt om ikke-root-brugere skal kunne fange pakker.
2. Tilføj din bruger til gruppen, der må fange trafik: `sudo usermod -aG wireshark $USER`.
3. Log ud og ind igen (eller genstart VM'en), så gruppemedlemskabet træder i kraft.
4. Åbn Wireshark, og vælg interface Loopback: `lo` som det, du fanger trafik på — det er her al trafik i denne opgave kører, i modsætning til et almindeligt netværkskort som `eth0`.

Test at du kan starte en capture på lo, før du går videre med resten af modulet — det er den mest almindelige showstopper i hele opgaven, og den er hurtig at rette, hvis du opdager den tidligt.

Opgaver

1. Byg en simpel TCP echo-server og en tilhørende klient i Python med socket-modulet, der sender og modtager tekst over 127.0.0.1.
2. Fang en forbindelse i Wireshark på interfacet lo, og identificér SYN, SYN-ACK, ACK samt afslutningen (FIN/RST) i capturen.
3. Beskriv kort forskellen på TCP og UDP, og hvorfor HTTP er bygget oven på TCP og ikke UDP.
4. Brug netstat eller ss (fx ss -tnp) til at vise din servers aktive forbindelse på loopback, mens en klient er tilsluttet.

Afleveringskrav for modulet

- Kildekoden til din TCP-klient og -server.
- Wireshark-screenshot af handshake med de tre pakker markeret.
- Kort skriftlig forklaring af TCP vs. UDP.

Modul 2: DNS — fra navn til adresse

Formål: At forstå og selv kunne udføre navneopløsning, samt konfigurere din egen lokale DNS-opløsning til et selvvalgt domænenavn.

Hvorfor det er relevant i en sikkerhedskontekst

- DNS er ofte usikret og et hyppigt mål for spoofing og cache poisoning, fordi mange systemer stoler blindt på svaret uden yderligere verifikation.
- DNS bruges aktivt af angribere til data-eksfiltrering og command-and-control-kommunikation (DNS tunneling), netop fordi DNS-trafik sjældent inspiceres nøje.
- DNS er det første skridt i næsten enhver netværksforbindelse — en forkert eller kompromitteret DNS-opsætning kan omdirigere brugere til ondsindede systemer, uden at de mærker noget.

Opgaver

1. Skriv et script, der selv konstruerer og sender en DNS-forespørgsel over UDP til en resolver (i stedet for at bruge et bibliotek), og parser svaret.
2. Konfigurer en lokal navneopløsning i din VM's `/etc/hosts`, så domænenavnet `network.test` peger på `127.0.0.1` — altså din egen VM, ikke en anden maskine.
3. Undersøg forskellige recordtyper (A, CNAME, MX, TXT, NS) for et rigtigt domæne med dig eller `nslookup`, og forklar kort formålet med hver.
4. Fang et DNS-opslag i Wireshark (interfacet, der matcher din udgående forbindelse, fx `eth0` for opslag mod en ekstern resolver), og identificér query, response, transaktions-ID og TTL.

Afleveringskrav for modulet

- Dit DNS-opslags-script med kommentarer.
- Screenshot af din lokale navneopløsning der virker (fx `ping network.test`, der svarer fra `127.0.0.1`).
- Wireshark-screenshot af en DNS query/response med de nævnte felter markeret.

Sådan bygger du DNS-opslagsscriptet i Python

Formålet er at sende en gyldig DNS-forespørgsel uden at bruge et bibliotek som `dnspython` — du skal selv bygge de rå bytes, en DNS-pakke består af. Det lyder mere skræmmende, end det er, hvis du deler det op i header og spørgsmål.

Transportlaget: DNS-forespørgsler sender du over UDP, ikke TCP — brug derfor `socket.socket(AF_INET, SOCK_DGRAM)`. Send din forespørgsel til en offentlig resolver (fx `8.8.8.8`) på port 53 med `.sendto(data, (resolver_ip, 53))`, og modtag svaret med `.recvfrom(bufstørrelse)`.

DNS-headeren (12 byte): Byg headeren med Pythons `struct`-modul. Den skal som minimum indeholde et transaktions-ID (2 byte, valgt af dig — brug det til at genkende dit eget svar), et flags-felt (2 byte, hvor du sætter 'recursion desired'), samt antal spørgsmål, svar, authority- og additional-records (hver 2 byte, hvor kun spørgsmål er 1 og resten er 0 i en forespørgsel).

Spørgsmålssektionen: Domænenavnet skal encode som en serie af længde-præfikserede labels — fx bliver 'example.com' til byten 7 efterfulgt af bogstaverne 'example', byten 3 efterfulgt af 'com', og til sidst en nul-byte som afslutning. Efter navnet følger to 2-byte-felter: forespørgselstype (1 for A-records) og klasse (1 for internet/IN).

Parsing af svaret: Som minimum skal du kunne læse antal svar (answer count) ud af headeren i det modtagne svar, og bekræfte at transaktions-ID'et matcher det, du selv sendte. Vil du gå videre, kan du også forsøge at udtrække selve IP-adressen fra svarets answer-sektion — det kræver, at du håndterer DNS' pointer-komprimering (navne kan genbruges via en reference tilbage i pakken i stedet for at blive gentaget), som er en god øvelse i sig selv, men ikke et krav for at bestå modulet.

Modul 3: HTTP — byg din egen webserver

Formål: At forstå HTTP-protokollen indefra ved selv at bygge en minimal webserver, der taler protokollen råt oven på din TCP-socket fra modul 1.

Hvorfor det er relevant i en sikkerhedskontekst

- Forståelse af headers og statuskoder er en forudsætning for websikkerhed generelt — fx korrekt brug af sikkerhedsheaders og fejlhåndtering, der ikke lækker unødigt information.
- At bygge en HTTP-parser selv gør det konkret, hvor let det er at overse fejl i input-håndtering — en central kilde til sårbarheder i virkelige webservere.
- HTTP sender alt i klartekst; at se det direkte i en pakke-capture gør risikoen ved ukrypteret trafik håndgribelig frem for teoretisk.

Opgaver

1. Byg videre på din TCP-server fra modul 1, så den kan parse en rå GET-request (metode, sti, headers) og returnere et korrekt formateret HTTP-response.
2. Udvid serveren til at understøtte mindst to statuskoder (fx 200 og 404) og en header efter eget valg (fx Content-Type).
3. Test din server med `curl -v http://127.0.0.1:8080/`, og forklar med egne ord, hvad hver del af outputet betyder.
4. Fang en request til din server i Wireshark på interfacet lo, og vis, at hele indholdet er læsbart i klartekst.

Afleveringskrav for modulet

- Kildekoden til din HTTP-server.
- Output fra `curl -v` med din egen forklaring.
- Wireshark-screenshot med det læsbare HTTP-indhold markeret.

Sådan er din HTTP-server bygget i Python

Du skal genbruge TCP-serveren fra modul 1 som fundament — HTTP er i bund og grund bare en aftalt tekstformatering oven på en almindelig TCP-forbindelse. Bind fortsat til 127.0.0.1, men brug en ny port (fx 8080), så den ikke kolliderer med din TCP-server fra modul 1.

Læsning og parsing af requesten: Når du modtager data med `.recv()`, får du en samling bytes. Afkod dem til tekst med `.decode()`, og split på linjeskift (`\r\n` er HTTP's officielle linjeskift, men `\n` virker ofte også i praksis). Den første linje er request-linjen og indeholder metode, sti og HTTP-version adskilt af mellemrum (fx `'GET / HTTP/1.1'`) — split den linje på mellemrum for at få de tre dele hver for sig. De efterfølgende linjer, frem til en tom linje, er headers på formen `'Navn: Værdi'` — split hver linje på `:` og gem dem i en dictionary, hvis du vil kunne slå dem op.

Opbygning af responset: Et HTTP-response starter med en statuslinje (fx `'HTTP/1.1 200 OK\r\n'`), efterfulgt af en eller flere headers (fx `'Content-Type: text/plain\r\n'` og `'Content-Length: <antal bytes i body>\r\n'`), en tom linje som markerer overgangen til indholdet, og til sidst selve body'en. Byg denne streng, encode den til bytes, og send den samlet med `.sendall()`.

Content-Length er ikke valgfri: Sæt altid en korrekt Content-Length-header, der matcher antallet af bytes i din body (ikke antallet af tegn — de kan afvige, hvis du bruger specialtegn). Mange HTTP-klienter, herunder browsere, bruger denne header til at vide, hvornår responset er færdigt.

Statuskoder: Implementér som minimum 200 OK (når stien findes) og 404 Not Found (når den ikke gør). Strukturér din kode, så det er let at tilføje flere statuskoder senere — fx ved at bruge en dictionary, der mapper statuskoder til deres tekst ('OK', 'Not Found' osv.).

Modul 4: HTTPS/TLS — kryptering af forbindelsen

Formål: At forstå, hvordan TLS beskytter HTTP-trafik mod aflytning, og selv kunne konfigurere det oven på din egen webserver.

Hvorfor det er relevant i en sikkerhedskontekst

- TLS er fundamentet for tillid på internettet i dag — uden det kan enhver på samme netværk læse eller ændre trafikken undervejs.
- Forkert konfigureret TLS (udløbne certifikater, forældede protokolversioner) er blandt de hyppigste fund i sikkerhedsscanninger af rigtige systemer.
- Certificate trust chain er det, der i praksis forhindrer man-in-the-middle-angreb — at forstå den er en forudsætning for at kunne vurdere, om en forbindelse reelt er sikker.

Opgaver

1. Generér et selvsigneret certifikat til dit domænenavn fra modul 2 med openssl.
2. Tilføj TLS til din HTTP-server ved at wrappe din eksisterende socket med Pythons ssl-modul.
3. Test forbindelsen med openssl `s_client -connect 127.0.0.1:8443`, og forklar outputtet: handshake, valgt cipher suite og certifikatkæde.
4. Fang den samme request som i modul 3, nu over HTTPS, i Wireshark — og sammenlign: hvad kan du se nu, som du kunne se før, og hvad kan du ikke længere se?

Afleveringskrav for modulet

- Dit selvsignerede certifikat (eller relevant udsnit) og den opdaterede serverkode.
- Output fra openssl `s_client` med din forklaring.
- Sammenligning af de to Wireshark-captures (HTTP vs. HTTPS) med konkrete observationer.

Sådan genererer du et selvsigneret certifikat

Kør følgende kommando i din VM for at generere et privat nøglepar og et selvsigneret certifikat i én omgang:

```
openssl req -x509 -newkey rsa:2048 -keyout key.pem -out cert.pem -days 365 -nodes -subj "/CN=network.test"
```

-nodes betyder, at den private nøgle ikke krypteres med en adgangskode (nødvendigt, for at din Python-server kan læse den uden manuel indtastning), og -subj "/CN=network.test" sætter certifikatets Common Name til det domænenavn, du valgte i modul 2 — det er netop denne værdi, browseren/klienten sammenligner med det domænenavn, den forbandt til.

Sådan tilføjer du TLS til din server i Python

SSLContext: Opret en `ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)`, og indlæs dit certifikat og din nøgle med `context.load_cert_chain(certfile="cert.pem", keyfile="key.pem")`.

Wrap din socket: Wrap din lyttende socket, før du kalder `.accept()`, med `context.wrap_socket(din_socket, server_side=True)`. Herefter opfører den wrappede socket sig præcis som en almindelig socket i resten af koden — `.accept()`, `.recv()` og `.sendall()` virker uændret, så du kan genbruge det meste af din HTTP-server fra modul 3 direkte.

Ny port: Brug en ny port til din HTTPS-server, fx 8443, så du stadig kan køre din almindelige HTTP-server på 8080 side om side og sammenligne dem direkte.

Modul 5: Integration — hele kæden samlet

Formål: At sætte DNS, TCP, HTTP og TLS sammen til én sammenhængende, dokumenteret rejse fra domænenavn til indhold i browseren.

Hvorfor det er relevant i en sikkerhedskontekst

- At kunne se hele kæden på én gang — ikke kun enkeltdelene — er forskellen på at kunne bruge protokollerne og reelt kunne fejlfinde eller vurdere sikkerheden i et system.
- De fleste sikkerhedsvurderinger og incident-undersøgelser kræver netop denne evne til at spore et forløb på tværs af flere lag i stakken.

Opgaver

1. Tilgå din server via dit domænenavn med `curl -v https://network.test:8443/ -k` (flaget `-k` springer certifikatvalidering over, hvilket er forventet med et selvsigneret certifikat), og verificér at hele kæden virker: DNS → TCP → TLS → HTTP.
2. Lav én samlet Wireshark-capture af hele forløbet, fra DNS-opslaget til det færdige HTTP-svar.
3. Lav et simpelt diagram eller en skriftlig gennemgang, der viser rækkefølgen af de fire trin, og hvad der sker i hvert af dem.

Dokumentationskrav for modulet

- Den samlede Wireshark-capture.
- `curl -v`-output der viser hele kæden, med din forklaring af hvert trin.

6. Samlet dokumentationskrav

- Al kildekode fra modul 1-4 (TCP-server/klient, DNS-opslagsscript, HTTP-server, TLS-udvidelse), kommenteret.
- En samlet README/dokumentation, der beskriver din opsætning og dine valg gennem alle moduler.
- Bevis-output for hvert modul (Wireshark-screenshots, kommandooutput) som angivet under hvert modul ovenfor.
- Den samlede end-to-end-dokumentation fra modul 5.